

StudioChat – Produktmappe

Spezifikation, Design-System, Roadmap und Pitch. Stand 8. August 2026.

Spezifikation

Stand 8. August 2026. Was das Produkt ist, für wen, und nach welchen Regeln es gebaut ist. Wer die App weiterentwickelt oder verkauft, sollte dieses Dokument gelesen haben.

1. Was es ist

StudioChat ist das interne Betriebssystem einer Studiokette. Nicht ein Messenger mit Zusatzfunktionen, sondern ein Werkzeug, das die immer gleichen Fragen eines Betreibers mit mehreren Standorten beantwortet:

Wo bleibt gerade etwas liegen? Was ist kaputt? Wer wartet auf mich?

Alles andere – Chat, Dateien, Schichtplan – ist Mittel zum Zweck.

Erster Kunde: Körperformen, 14 EMS-Studios im Raum Köln/Bonn.

Der Unterschied zu WhatsApp

Die ehrliche Konkurrenz ist keine Software, sondern eine WhatsApp-Gruppe. Was die nicht kann:

	WhatsApp-Gruppe	StudioChat
Aufgaben mit Frist und Nachweis	-	✓
Wiederkehrende Aufgaben (Putzplan)	-	✓
Materialbestand, Einkaufsliste, Bestellmail	-	✓
Gerätehistorie („3× defekt in 90 Tagen“)	-	✓
Schichttausch mit Freigabe	-	✓
Urlaubsantrag über alle Standorte	-	✓
Ablaufende Nachweise (Erste Hilfe, Lizenzen)	-	✓
Ausgeschiedene Mitarbeiter lesen nicht mit	-	✓
„Wo bleibt etwas liegen?“	-	✓

Umgekehrt kann WhatsApp eines besser: **jeder hat es schon**. Deshalb muss StudioChat sich beim Chat an WhatsApp messen lassen – ein Chat, der sich schlechter anfühlt, kostet die ganze Einführung.

2. Rollen

Drei Rollen, kein Rechte-Baukasten. Mehr Rollen hieße: jemand muss sie pflegen.

Rolle	Sieht	Darf zusätzlich
Mitarbeiter	seine Studios	abhaken, melden, eintragen, schreiben
Studio-Leiter	seine Studios	Aufgaben erteilen, Schichten planen, Urlaub genehmigen, Geräte pflegen
Chef	alles	Zugänge und Rollen vergeben, Nachweise führen, System

Grundsätze, die nicht verhandelbar sind:

- Selbstregistrierung ergibt **immer** die Rolle Mitarbeiter.
- Niemand ändert seine eigene Rolle oder Studio-Zuordnung.
- Niemand genehmigt sich selbst Urlaub.
- Nachweise sehen nur die betroffene Person und der Chef – die Studio-Leitung ausdrücklich nicht.
- Direktnachrichten liest niemand außer den zwei Beteiligten, auch der Chef nicht.

3. Datenmodell

Firestore, Region `europa-west1` (Belgien).

```

`` studios/{studioKey}/ studioKey = "studio-" + Listenplatz todos/ Aufgaben (auch aus Geräte-
Meldungen) cleaning/ Putzplan-Aufgaben cleaningNotes/ Notizen zum Putzplan devices/ Geräte mit
Zustand deviceLog/ Meldungen je Gerät (unveränderlich) shifts/ Schichten inkl. Tausch-Zustand
absences/ Abwesenheiten mit Antrags-Status handovers/ Übergaben an die nächste Schicht
channels/{kanal}/messages/ Chat, "allgemein" + je Studio dms/{paarId}/messages/
Direktnachrichten announcements/ Infos der Leitung mit Lesebestätigung board/ Schwarzes Brett
(studioübergreifend) documents/ + documentData/ Metadaten getrennt vom Dateiinhalt
inventory/{studioKey} Material – EIN Dokument je Studio certificates/ Nachweise archives/{jahr-
KWnn} Wochen-Sicherungen für die Vorhersage users/, pushTokens/, config/ Konten, Geräte,
Einstellungen
``

```

Die eine Regel, die man kennen muss

Der Studio-Schlüssel ist der Listenplatz, nicht der Name. `studioKey('Hürth') === 'studio-6'`, weil Hürth der siebte Eintrag in `KONFIG.studios` ist.

Wer die Liste umsortiert oder einen Eintrag löscht, ordnet ****allen**

bestehenden Daten ein anderes Studio zu. **** Neue Studios ausschließlich**

hinten anhängen. Ein geschlossenes Studio bleibt in der Liste stehen.

Warum Material ein einziges Dokument je Studio ist

22 Artikel × 14 Studios als Einzeldokumente wären 308 Dokumente und ebenso viele Schreibvorgänge beim Zählen. Als ein Dokument je Studio: 14 Dokumente, ein Schreibvorgang je Studio, verzögert gebündelt. **Preis dafür:** die Sicherheitsregeln können „Soll setzen“ nicht von „Ist

eintragen" unterscheiden – beides steht im selben Dokument. Diese Trennung ist eine Regel der Oberfläche, keine des Servers. Bewusst so entschieden.

4. Sicherheit

Die Rechte werden auf dem Server geprüft, nicht in der App. Die Sicherheitsregeln (390 Zeilen) sind die eigentliche Grenze; was in der Oberfläche versteckt ist, ist zusätzlich in der Datenbank gesperrt.

Bekannte Ausnahmen, ehrlich benannt:

Stelle	Warum die Oberfläche entscheidet
Material: Soll-Menge und Löschen	ein Dokument je Studio, siehe oben
Aussehen und Meldungen	reine Geräte-Einstellungen, kein Serverbezug

Weiteres:

- **Firestore-Listen** werden komplett abgelehnt, wenn auch nur ein zurückgegebenes Dokument unlesbar wäre. Jede Abfrage muss deshalb selbst filtern – das ist kein Stilmittel, sondern Pflicht.
- **Bilder und Sprachnachrichten** aus der Datenbank laufen durch `safeMedia()`, Links durch `safeUrl()`. Ohne diese Prüfung wäre ein gespeichertes `javascript:`-Feld ein Einfallstor.
- **Gelöschtes** liegt 30 Tage im Papierkorb.
- **Nächtliche Sicherung** um 02:40, sieben Tage Aufbewahrung.
- **Es gibt keinen Zugangscode**, mit dem man sich zum Chef machen könnte.

5. Automatische Abläufe

20 Cloud Functions, Region `europa-west1`.

Wann	Was
07:30	Erinnerung an heute fällige Aufgaben
08:00	Geburtstagsgruß im Chat
08:15	Nachweise, die in 60/14/0 Tagen ablaufen
02:40	vollständige Sicherung der Datenbank (7 Tage)
03:15	erledigte einmalige Putzaufgaben endgültig entfernen
03:30	Papierkorb älter als 30 Tage löschen
1. des Monats	Monatsbericht per E-Mail an den Chef
wöchentlich	Material und Putzplan ins Archiv sichern
bei Ereignis	Push-Nachrichten, Termin-Mails

6. Technische Entscheidungen und ihr Preis

Entscheidung	Vorteil	Preis
Eine HTML-Datei (11.474 Zeilen)	kein Build, kein Werkzeugkasten, in fünf Jahren noch lesbar	keine Modularisierung; Suchen statt Springen
Kein Framework	keine Abhängigkeit, die veraltet; 372 Funktionen in reinem JavaScript	mehr Handarbeit bei der Darstellung
Alles im Speicher rechnen	sofortige Reaktion, keine Serverkosten	ab etwa 40 Studios neu zu denken
Ein Beobachter je Studio	Live-Aktualisierung überall	14 Verbindungen beim Chef
Dateien als Text in der Datenbank	kostenlos	Grenze bei ~0,7 MB je Datei
Alles auf Deutsch, auch im Code	der Kunde kann mitlesen	keine internationale Weitergabe ohne Übersetzung
Firebase-Gratisstufe	keine laufenden Kosten	Kontingente statt Skalierung

7. Was das Produkt bewusst nicht tut

Wichtig für Gespräche mit Mitarbeitern und Mitarbeitervertretung – und ein Verkaufsargument, kein Mangel:

- **Keine Standortverfolgung.**
- **Keine Arbeitszeiterfassung**, keine Stempeluhr.
- **Keine Leistungsmessung**, kein Ranking zwischen Mitarbeitern.
- **Kein Mitlesen von Direktnachrichten** – auch nicht durch den Chef.
- **Keine KI-Auswertung**, keine Weitergabe an Dritte.
- **Keine Werbung.**

Nicht gebaut, weil es der Sache schadet:

- **„Schreibt gerade ...“** – eine Schreiboperation je Tastenanschlag und Person. Teuerster Effekt, geringster Nutzen.
- **Lesebestätigung im Teamchat** – wäre eine Anwesenheitskontrolle.
- **Urlaubskonto mit Resttagen** – das ist Lohnbuchhaltung, halb gebaut schlimmer als gar nicht.
- **Automatische Schichtplanung** – wer wann kann, hängt an Absprachen, die nicht in der App stehen.
- **Aufgaben per Ziehen umsordieren** – die Reihenfolge ergibt sich aus Frist und Dringlichkeit.

8. Grundsätze der Oberfläche

Diese fünf Sätze erklären neun von zehn Gestaltungsentscheidungen:

1. **Jede Seite beantwortet ihre Frage im ersten Bildschirm.** Was seltener gebraucht wird, ist zugeklappt und einen Tipp entfernt.
2. **Lesen kommt vor Schreiben.** Listen stehen vor Formularen.
3. **Dringlichkeit schlägt Alphabet** – außer da, wo die Reihenfolge etwas Körperliches abbildet (die Materialliste ist der Weg durchs Lager).
4. **Ein Fingerziel ist mindestens 44 Pixel hoch und beschriftet.**
5. **Nichts wird zweimal angeboten.** Zwei Wege zum selben Ort machen eine Oberfläche nicht bedienbarer, sondern unübersichtlicher.

9. Umfang

Oberfläche	11.474 Zeilen (8.395 JavaScript, 1.751 CSS)
Funktionen im Frontend	372
Cloud Functions	20
Sicherheitsregeln	390 Zeilen
Automatische Durchläufe	29 Dateien
Handbuch	23 Kapitel, 15 Seiten als PDF
Betriebskosten	0 € (Firebase-Gratisstufe)

Design-System

Stand 8. August 2026. Die Werte hier sind **aus dem Code ausgelesen**, nicht erfunden. Wer etwas Neues baut, findet hier die Bausteine, damit es nicht nach einem Fremdkörper aussieht.

1. Farben

Alle Farben liegen als CSS-Variablen in `:root` (dunkel) und `body.light` (hell). **Nie eine Farbe direkt schreiben** – sonst fehlt sie im anderen Modus.

Flächen

Marke	Dunkel	Hell	Wofür
--bg	#12131C	#F4F6FA	Seitengrund
--bg-2	#1C1E2A	#FFFFFF	Karten, Fenster, Blasen
--bg-soft	#171925	#EDF0F6	Eingabefelder, Reiterleisten
--bg-3	#252838	#E3E8F1	betonte Flächen
--line	weiß 9 %	schwarz 10 %	ruhige Trennlinie
--line-2	weiß 17 %	schwarz 18 %	Rahmen von Bedienelementen

Das Dunkel ist ein **Schiefer-Blau**, kein Schwarz: sachlich und über lange Zeit angenehmer zu lesen als harter Neon-Kontrast.

Marke

Marke	Dunkel	Hell	Wofür
--accent	#38BDF8	#0369A1	primär, Cyan aus dem Logo
--accent-2	#A78BFA	#6D28D9	Violett, zweite Ebene
--accent-3	#F472B6	#BE185D	Pink, Akzent
--brand	Verlauf Violett → Cyan → Pink, 135°		
--on-accent	#0A1420	#FFFFFF	Text auf Akzentflächen

Die hellen Töne sind bewusst **dunkler als die dunklen** – auf Weiß wäre `#38BDF8` nicht lesbar.

Bedeutung

Marke	Wert	Bedeutung
--warm	#FBBF24	Achtung: fehlt, läuft ab, wartet
--danger	#FB4E6D	Fehler, defekt, überfällig, löschen
--accent-d	#38BDF8 / #075985	Erfolg, erledigt

Regel: Rot heißt „kaputt oder zu spät“. Gelb heißt „bald oder wartet“. Nie Rot für „wartet auf Entscheidung“ – ein Urlaubsantrag ist kein Defekt.

Text

--text (Hauptton) · --text-2 (Nebentext) · --text-3 (Metadaten). Drei Stufen, mehr nicht.

2. Schrift

Marke	Familie	Wofür
<code>--font-head</code>	Barlow Condensed, 500–800	Überschriften, Zahlen, Marken
<code>--font-body</code>	Barlow, 300–700	alles andere

Größen (aus dem Code):

Element	Größe
Seitentitel <code>h2</code>	<code>clamp(1.4rem, 3.5vw, 1.9rem)</code>
Kartentitel <code>h3</code>	~1,05 rem
Fließtext	0,92–0,97 rem
Nebentext	0,84–0,90 rem
Metadaten	0,72–0,78 rem
Marken/Abzeichen	0,62–0,74 rem, Großbuchstaben, <code>letter-spacing: .5px</code>

Die Schriftgröße ist über die Einstellungen in drei Stufen skalierbar – neue Größen deshalb **relativ** (`rem`), nie in Pixeln.

3. Maße

Marke	Wert	Wofür
<code>--radius</code>	<code>18px</code>	Karten
<code>--radius-lg</code>	<code>26px</code>	große Flächen
—	<code>999px</code>	Chips, Marken, runde Knöpfe
—	<code>9–14px</code>	Eingabefelder, kleine Knöpfe

Seitenrand: `clamp(14px, 4vw, 28px)` – auf dem Handy schmal, am Rechner großzügig. Dazu immer die Geräteänder addieren: `padding-left: calc(clamp(14px, 4vw, 28px) + var(--sal))`. Ohne `--sat/--sab/--sal/--sar` liegen Knöpfe am iPhone unter der Statusleiste und lassen sich nicht antippen.

Abstände: 6 · 8 · 10 · 14 · 18 · 22 px. Keine Zwischenwerte erfinden.

4. Fingerziele — die harte Regel

Alles, was man antippt, ist mindestens 44 Pixel hoch.

Das war der häufigste Fund im gesamten Audit. Gefunden und behoben:

Ort	vorher	jetzt
Werkzeuge an einer Chat-Nachricht	21 × 19	Blatt-Einträge 48
Werkzeuge an einer Aufgabe	25 × 27	Blatt-Einträge 48
Löschen am Nachweis	21 × 25	36 × 44
Studio-Knöpfe „Wo etwas defekt ist“	-	42

Ausnahme: Symbole **innerhalb** einer Zeile, die selbst anklickbar ist (die Kamera in der Aufgaben-Fußzeile). Dort ist die ganze Zeile das Ziel.

5. Bewegung

Marke	Kurve	Wofür
--ease-out	cubic-bezier(.16,1,.3,1)	Einblenden, Ansichtswechsel
--ease-ios	cubic-bezier(.32,.72,0,1)	Gleiten, Aufklappen
--spring	cubic-bezier(.34,1.42,.64,1)	Druck-Rückmeldung, Pop

Dauern: 180 ms (Rückmeldung) · 260–350 ms (Blätter, Aufklappen) · 460–500 ms (Fenster von unten).

@media (prefers-reduced-motion: reduce) schaltet alles auf 0,01 ms. Neue Animationen brauchen dort keine Sonderbehandlung, sie sind mit erfasst.

6. Bausteine

Karte `.card` Grundfläche für alles. Mit `data-fold="name"` wird sie aufklappbar; der Zustand merkt sich das Gerät in `PREFS.folds`. `data-fold-offen="1"` startet offen. Wann zuklappen: wenn die Karte ein Formular ist oder ein Beleg zu der Karte darüber. Nie die Karte, die die Frage der Seite beantwortet.

Aktionsblatt `.msg-sheet` Gleitet von unten, dunkler Grund dahinter. Kopf mit Bezug (wer/was/wann), optional eine Reaktionsreihe, dann `.ms-act`-Einträge à 48 Pixel mit Symbol und Wort, zuletzt „Abbrechen“. Benutzt von: Chat-Nachricht, Aufgabe, Dokument. Wann: sobald es mehr als zwei Aktionen an einem Listeneintrag gibt.

Studio-Übersicht `.dev-wo` Reihe anzutippender Studios mit Zahl, darüber eine `.sec-head`-Überschrift und ein erklärender Satz. Rot = defekt, gelb = wartet. Benutzt von: „Wo etwas los ist“ (Start), „Wo etwas defekt ist“ (Geräte), „Wartet

auf deine Entscheidung" (Team). Wann: immer, wenn eine Seite nur ein Studio zeigt, das Problem aber woanders liegen kann.

Chip-Reihe `.chip-row` / `.sort-row` Eine Zeile, seitlich schiebbar, nie umbrechend. Zahl am Chip über `.chip-num`. Warum nicht umbrechen: vier Filter in drei Zeilen sind 150 Pixel über einer Liste mit drei Einträgen.

Leerer Bereich `emptyHTML(titel, text)` Muss den Grund und den Ausweg nennen. Nicht „Keine Aufgaben“, sondern „Keine dir zugewiesenen Aufgaben - es gibt 4, tippe auf ‚Alle‘“.

Rückgängig `offerUndo(text, fn)` Acht Sekunden. Für alles, was löscht. Zusätzlich `confirm()`, wenn die Wirkung über den eigenen Bildschirm hinausgeht (14 Studios, alle Kollegen).

7. Rollen in der Oberfläche

Attribut	Sichtbar für
<code>data-chef-only="1"</code>	nur Chef
<code>data-manage-only="1"</code>	Chef und Leiter
(nichts)	alle

Beides wird in `applyRoleVisibility()` gesetzt. **Verlass dich darauf nie allein** – die Sicherheitsregeln müssen dasselbe sagen.

8. Sprache

- **Deutsch, ohne Fachwörter.** „Fehlt“ statt „Differenz“, „Wo etwas los ist“ statt „Priorisierung“.
- **Du**, nicht Sie.
- **Knöpfe sagen, was passiert:** „Defekt melden“, nicht „Absenden“.
- **Meldungen nennen das Ding beim Namen:** „EMS-Gerät 2 ist defekt“, nicht „1 Gerät defekt“.
- **Zahlen vor Adjektiven:** „3 Artikel fehlen“, nicht „einige Artikel fehlen“.
- Auch **Kommentare im Code sind auf Deutsch** und erklären das *Warum*, nicht das *Was*.

9. Prüfliste für neue Oberfläche

Vor dem Einchecken durchgehen:

- ☐ Beantwortet die Seite ihre Frage im ersten Bildschirm?
- ☐ Steht die Liste vor dem Formular?
- ☐ Ist jedes Fingerziel ≥ 44 Pixel hoch und beschriftet?

- [] Nur Farbvariablen benutzt, keine festen Werte?
- [] Funktioniert es in hell **und** dunkel?
- [] Sagt der leere Zustand Grund und Ausweg?
- [] Hat alles Löschende eine Rückfrage oder Rückgängig?
- [] Sind die Geräteränder (`--sa*`) berücksichtigt?
- [] Bei 390 Pixeln Breite gemessen – nicht geschätzt?
- [] Gibt es die Funktion woanders schon? Dann dort verlinken, nicht nachbauen.

Roadmap

Stand 8. August 2026. Was als Nächstes kommt, in der Reihenfolge, in der es sich lohnt – und was es kostet.

Die zehn Bereiche des Master-Audits sind durch. Alles hier ist **Ausbau**, nichts davon ist nötig, damit die App im Alltag funktioniert.

Phase 0 — Was jetzt sofort ansteht (du)

Zwei Handgriffe, beide einmalig, beide kostenlos. Anleitung in `DEIN-TEIL.md`.

Was	Warum	Dauer
<code>MATERIAL-SHEETS.gs</code> in Apps Script einfügen	die Google-Tabelle läuft sonst mit der alten, langsamen Fassung	5 Min
Export-Rolle fürs Dienstkonto setzen	ohne sie läuft die nächtliche Sicherung ins Leere	3 Min

Phase 1 — Bevor ein zweiter Kunde kommt

Alles kostenlos, alles ohne neue Abhängigkeit.

#	Was	Warum	Aufwand
1.1	Eigene Absenderadresse für E-Mails	Berichte kommen von einer Gmail-Adresse; bei einem fremden Kunden wirkt das nicht wie ein Produkt	klein, aber Domain nötig
1.2	Einrichtungs-Assistent beim ersten Start	heute wird <code>konfig.js</code> von Hand bearbeitet. Ein geführter Ablauf (Firma, Studios, Firebase-Daten) macht aus „ich richte das ein“ ein „der Kunde richtet das ein“	mittel
1.3	Wischen zum Abhaken auch im Putzplan prüfen	funktioniert bei Aufgaben bestätigt; im Putzplan nur im Code gleich, nie am Gerät geprüft	klein
1.4	Fehlerbericht bei Abstürzen	heute merkt niemand, wenn bei einem Mitarbeiter etwas nicht lädt	klein

Phase 2 — Wenn die App wachsen soll

Ab hier wird es an einzelnen Stellen kostenpflichtig. Jede Position nennt den Preis.

#	Was	Warum	Kosten
2.1	Echter Dateispeicher statt Datenbank	hebt die 0,7-MB-Grenze für Dokumente auf	Firebase Storage, ab ~0,03 €/GB/Monat
2.2	Serverseitige Filter statt „alles im Speicher“	die Grenze liegt bei etwa 40 Studios oder einigen hundert Aufgaben je Studio	Entwicklungsaufwand, keine laufenden Kosten
2.3	Sammel-Dokument für Studio-Zahlen	die drei Übersichten („wo etwas los ist“, „wo etwas defekt ist“, „wartet auf dich“) fragen heute je Studio einzeln ab	Entwicklungsaufwand
2.4	Volltextsuche über den ganzen Verlauf	heute findet die Suche den offenen Kanal vollständig und das Neueste der anderen	Suchdienst, ab ~20 €/Monat
2.5	Kanalauswahl mit Suchfeld	ab etwa 25 Studios trägt die waagerechte Leiste nicht mehr	klein

Phase 3 — Mehrere Kunden in einer App

Erst ab dem fünften bis sechsten Kunden sinnvoll. Vorher ist ein eigenes Firebase-Projekt je Kunde einfacher, sicherer und billiger.

#	Was	Warum
3.1	Mandantenfähigkeit	eine Datenbank für alle Kunden, getrennt über eine Firmen-Kennung. Betrifft jede Abfrage und jede Sicherheitsregel
3.2	Abrechnung	Pläne, Rechnungen, Zahlungsanbieter
3.3	Verwaltungsoberfläche für dich	Kunden anlegen, Kontingente sehen

Warnung: Phase 3 ist kein Ausbau, sondern ein Umbau. Wer sie zu früh beginnt, verlangsamt jede weitere Änderung. Bis dahin gilt: **ein Kunde = ein Firebase-Projekt = eine `konfig.js`** .

Was bewusst nicht auf der Roadmap steht

Idee	Warum nicht
Native App im App Store	Die PWA leistet dasselbe. Ein Store-Eintrag heißt: Konten, Gebühren, Prüfverfahren, zwei zusätzliche Fassungen zu pflegen
KI-Auswertung der Chats	Datenschutz zuerst, und die Fragen eines Studios lassen sich rechnen
Zeiterfassung	Steht im Handbuch unter „was die App bewusst nicht tut“. Das ist ein Versprechen an die Mitarbeiter
Kundenverwaltung / Termine	Dafür haben die Studios ihr Kassensystem. Zwei Systeme mit denselben Daten laufen auseinander
Freie Rollen und Rechte	Drei Rollen kann man erklären. Ein Rechte-Baukasten muss gepflegt werden

Reihenfolge, wenn Zeit knapp ist

1. **Phase 0** – zwei Handgriffe, sonst laufen Sicherung und Tabelle nicht richtig.
2. **1.1 eigene Absenderadresse** – die einzige Stelle, an der die App heute noch nach Basterei aussieht.
3. **1.2 Einrichtungs-Assistent** – macht aus einem Projekt ein Produkt.
4. Alles andere nach Bedarf.

SWOT und Pitch

Stand 8. August 2026.

Der Pitch in einem Satz

**StudioChat beantwortet die einzige Frage, die ein Studiobetreiber jeden

Morgen hat: Wo bleibt gerade etwas liegen?**

In dreißig Sekunden

Wer mehrere Studios betreibt, führt sie über eine WhatsApp-Gruppe. Aufgaben verschwinden im Verlauf, ein defektes Gerät steht zwischen Urlaubsgrüßen, und niemand weiß, in welchem Studio die Handtücher ausgehen. Ausgeschiedene Mitarbeiter lesen mit.

StudioChat ersetzt diese Gruppe durch ein Werkzeug, das den Betrieb kennt: Aufgaben mit Frist und Foto-Nachweis, Putzplan mit Wiederholung, Materialbestand mit Einkaufsliste und fertiger Bestellmail,

Gerätehistorie mit Wiederholungstäter-Warnung, Schichttausch mit Freigabe, Urlaubsanträge über alle Standorte.

Und auf jedem Bildschirm steht oben, **wo** gerade etwas hakt.

Läuft im Browser, installiert sich wie eine App, kostet im Betrieb nichts.

In zwei Minuten (Vorführung)

1. **Startseite als Chef.** „Wo etwas los ist“ – vier Studios, gewichtet nach Dringlichkeit. Antippen → man ist im Studio.
2. **Aufgaben.** Überfälliges steht oben, Studios mit Problemen zuerst. Abhaken durch Wischen, Foto als Nachweis.
3. **Geräte.** „EMS-Gerät 2 ist defekt · ansehen“. Verlauf: *3× defekt in 90 Tagen*. Eine Meldung erzeugt automatisch eine Aufgabe – aber nur eine.
4. **Material.** „3 Artikel fehlen · nur diese zeigen“. Einkaufsliste über alle Studios, Bestellmail fertig vorbereitet.
5. **Team.** „Wartet auf deine Entscheidung: Hürth 1“ → Urlaubsantrag, genehmigen.

Fünf Bildschirme, fünf Antworten, keine Schulung nötig.

SWOT

Stärken

- **Löst ein echtes, teures Problem.** Ein ausgefallenes EMS-Gerät kostet Termine. Ein vergessener Erste-Hilfe-Kurs kostet mehr.
- **Der „Wo?“-Blick** – auf Startseite, bei Geräten und im Team dieselbe Form. Kein Messenger hat das, und kein allgemeines Aufgabenwerkzeug kennt „Studio“.
- **Betriebskosten null.** Firebase-Gratisstufe trägt 14 Studios mühelos.
- **Vollständig auf Deutsch**, auch im Code – der Kunde kann mitlesen.
- **Ein neuer Kunde = eine Datei.** `konfig.js` austauschen, fertig.
- **29 automatische Durchläufe** – jede Änderung wird durchgeklickt, bevor sie live geht.
- **Datenschutz als Verkaufsargument:** keine Ortung, keine Zeiterfassung, keine Leistungsmessung, kein Mitlesen. Das steht so im Handbuch und ist in Gesprächen mit Mitarbeitern Gold wert.
- **15-seitiges Handbuch**, das ein Mensch lesen kann.

Schwächen

- **Eine Datei mit 11.474 Zeilen.** Wartbar, solange eine Person sie kennt. Zu zweit wird es eng.
- **Skalierbarkeit ist gedeckelt.** Alles wird im Speicher gerechnet, ein Beobachter je Studio. Ab etwa 40 Studios muss umgebaut werden.
- **Dateien liegen als Text in der Datenbank**, Grenze ~0,7 MB. Größeres nur als Link.
- **Kein Einrichtungs-Assistent.** Ein neuer Kunde braucht jemanden, der `konfig.js` und Firebase versteht.
- **E-Mails kommen von einer Gmail-Adresse.**

- **Ein Kunde.** Alle Annahmen stammen aus einem Betrieb.
- **Bus-Faktor 1.**

Chancen

- **Franchise-Ketten.** EMS-Studios sind fast immer Ketten – wer eines überzeugt, bekommt oft mehrere.
- **Übertragbar.** Physiotherapie, Friseure, Fahrschulen, Bäckereien: überall dieselbe Struktur – mehrere Standorte, Schichtbetrieb, Material, Geräte, Nachweispflichten. Nur `konfig.js` und ein paar Wörter ändern sich.
- **Nachweispflichten wachsen.** Dokumentation ist ein Argument, das von selbst stärker wird.
- **Der Preis ist konkurrenzlos.** Wettbewerber verlangen pro Kopf und Monat; hier liegen die Betriebskosten bei null.

Risiken

- **WhatsApp ist umsonst und schon installiert.** Die Einführung scheitert nicht an Funktionen, sondern an Gewohnheit. Deshalb muss der Chat mithalten – das war der Grund für Bereich 3.
- **Ein einzelner schlechter Tag** (Push kommt nicht an, App lädt nicht) wirft das Team zurück in die Gruppe.
- **Firebase-Kontingente.** Kostenlos, bis es das nicht mehr ist. Bei 14 Studios weit entfernt, aber es gibt keine Warnung, bevor es eng wird.
- **Abhängigkeit von Google.** Auth, Datenbank, Push, Hosting, Functions, Tabellen – alles aus einem Haus.
- **Als Produkt fehlt der Rahmen:** Vertrag, Auftragsverarbeitung, Verfügbarkeitszusage, Unterstützung. Für den eigenen Betrieb egal, für einen fremden Kunden nicht.

Was ein Käufer als Erstes fragen wird

Frage	Ehrliche Antwort
„Was kostet der Betrieb?“	Bei eurer Größe nichts. Firebase-Gratisstufe.
„Was, wenn ihr wegfällt?“	Der Quelltext ist eine Datei, alles auf Deutsch kommentiert. Die Daten liegen in eurem eigenen Firebase-Projekt.
„Können meine Leute mitgelesen werden?“	Direktnachrichten nein, auch vom Chef nicht. Steht im Handbuch.
„Was ist mit Arbeitszeiterfassung?“	Gibt es bewusst nicht.
„Wie lange dauert die Einführung?“	Zugänge anlegen, Studios eintragen, loslegen. Keine Schulung nötig – aber jemand muss Firebase einrichten.
„Wie viele Studios verträgt das?“	Bis etwa 40 wie gebaut. Darüber ist ein Umbau nötig, der bekannt und beschrieben ist.
„Wer hat es sonst noch?“	Ein Betrieb mit 14 Studios, seit Sommer 2026 im Einsatz.

Preisidee (unverbindlich)

Die Betriebskosten sind null; der Preis bildet Arbeit und Betreuung ab.

Modell	Preis	Enthält
Einrichtung	einmalig	Firebase-Projekt, <code>konfig.js</code> , Zugänge, Einweisung
Betreuung	monatlich je Betrieb, nicht je Kopf	Aktualisierungen, Fehlerbehebung, Sicherung
Eigenbetrieb	einmalig	Quelltext und Handbuch, Betrieb beim Kunden

Nicht pro Kopf abrechnen. Ein Studio mit 20 Aushilfen zahlt sonst mehr als eines mit fünf Vollzeitkräften – bei gleichem Nutzen. Und es bestraft genau das, was man will: dass alle die App benutzen.
